

Mapping Object Relationships : Code-Along (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-v2-orm-relationships>  <https://github.com/learn-co-curriculum/python-p3-v2-orm-relationships/issues/new>

Learning Goals

- Implement ORM methods to manage a one-to-many relationship between classes.
-

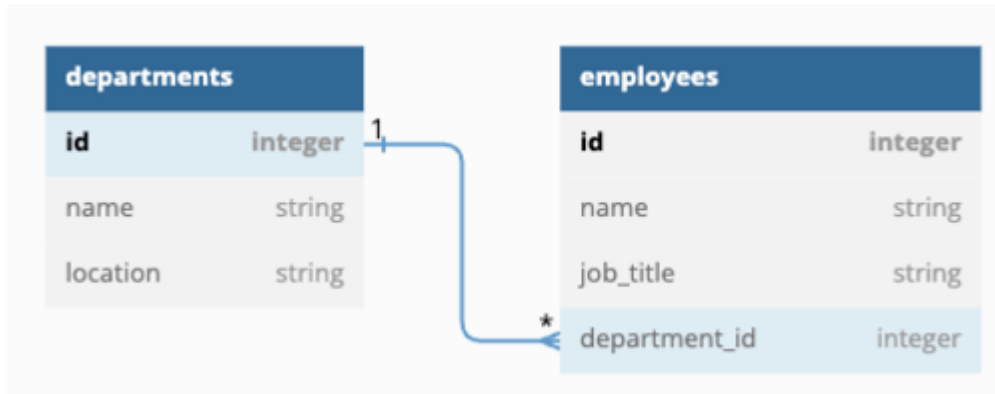
Key Vocab

- **Object-Relational Mapping (ORM)**: a programming technique that provides a mapping between an object-oriented data model and a relational database model.
-

Introduction

In this lesson, we'll implement a one-to-many relationship between departments and employees:

- An employee works in one department.
- A department has many employees.



Let's assume each employee works for exactly one department. The relationship represents composition since an employee must be associated with a department.

Code Along

This lesson is a code-along, so fork and clone the repo.

NOTE: Remember to run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```

pipenv install
pipenv shell
  
```

The `lib` folder contains starter code for the `Department` and `Employee` classes. Both classes include ORM methods for mapping the individual class to a corresponding table in the database:

Python Class	Relational Database Table
Department	departments
Employee	employees

The one-to-many relationship between `Department` and `Employee` is **not** implemented in the starter code. In this lesson, we will:

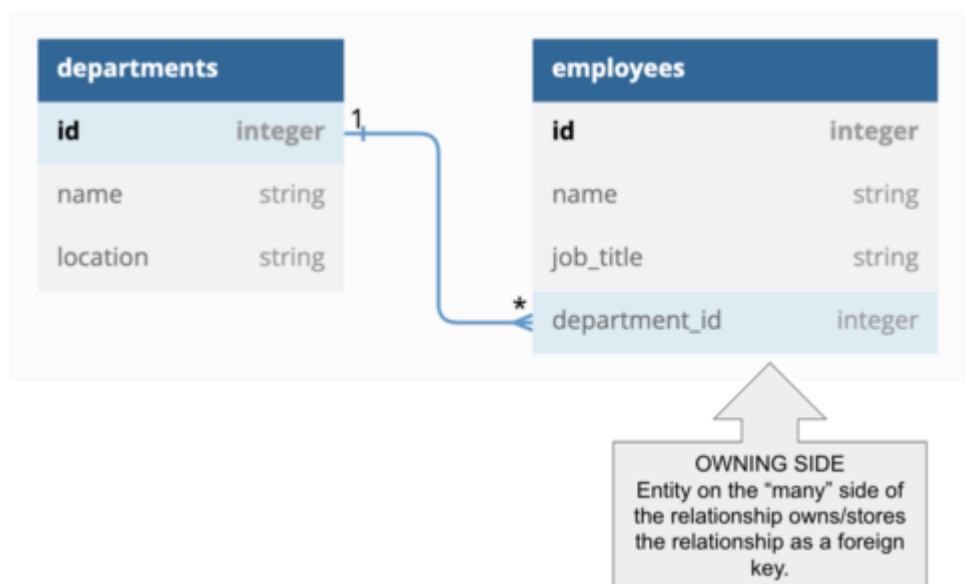
- Update the `Employee` class to add a new attribute to model the one-to-many relationship.

- Update the **Employee** class methods to store and manage the relationship in the "employees" database.
- Update the **Department** class to add a new method to **compute** (not store) a list of associated **Employee** instances.

Mapping a one-to-many relationship

The concept of **single source of truth** states we should design our software and data to avoid redundancy. In terms of relational database design, we want to store the relationship between two entities in just one place within the database. We do this to avoid issues that often arise when redundant data is not updated in a consistent manner.

When we have a relationship between two classes, we need to figure out where to store the relationship. For the one-to-many relationship, the class (or entity) on the "many" side is the owner of the relationship and is responsible for storing the relationship. Why? Beside each object on the "many" side (i.e. Employee) is related to just one entity on the "one" side (i.e. Department), thus each employee only needs to store a single value to maintain the relationship with its department.



Employee is on the many side of the relationship, thus it **owns** and manages the relationship by storing a single piece of data about the associated **Department**.

- The "Employee" class will have an attribute `department_id` to reference the id of the associated `Department` object.
- The "employees" table will contain a column `department_id` to store a foreign key reference to the "departments" table.

`Department` is on the "one" side of the relationship, thus it should not **store** a list of the associated `Employee` instances. Instead, a list of the many `Employee` objects associated with the department will be **computed** by looking at the `department_id` foreign key value stored with each employee.

Updating the `Employee` class to store the relationship

Let's update the `Employee` class to store the relationship to `Department` by making the following changes:

- Import the `Department` class.
- Update the `__init__` method to add a parameter `department_id` and store the value in a new attribute with the same name.
- Update the `__repr__` method to include the value of the new attribute.

```
from __init__ import CURSOR, CONN
from department import Department
```

```
class Employee:
```

```
    # Dictionary of objects saved to the database.
    all = {}
```

```
    def __init__(self, name, job_title, department_id, id=None):
        self.id = id
        self.name = name
        self.job_title = job_title
        self.department_id = department_id
```

```
    def __repr__(self):
        return (
```

```

        f"<Employee {self.id}: {self.name}, {self.job_title}, " +
        f"Department ID: {self.department_id}>"
    )

```

```

# existing ORM methods ...

```

Next, we'll update the `create_table` method to add a column named `department_id` to store the relationship as a foreign key in the "employees" table:

```

@classmethod
def create_table(cls):
    """ Create a new table to persist the attributes of Employee instances """
    sql = """
        CREATE TABLE IF NOT EXISTS employees (
            id INTEGER PRIMARY KEY,
            name TEXT,
            job_title TEXT,
            department_id INTEGER,
            FOREIGN KEY (department_id) REFERENCES departments(id))
    """
    CURSOR.execute(sql)
    CONN.commit()

```

We need up adapt the `save` , `update` , and `create` methods to persist the new relationship by storing the department id in the "employees" table:

```

def save(self):
    """ Insert a new row with the name, job title, and department id values of the current Employee object.
    Update object id attribute using the primary key value of new row.
    Save the object in local dictionary using table row's PK as dictionary key"""
    sql = """
        INSERT INTO employees (name, job_title, department_id)
    """

```

```

        VALUES (?, ?, ?)

    """

    CURSOR.execute(sql, (self.name, self.job_title, self.department_id))
    CONN.commit()

    self.id = CURSOR.lastrowid
    type(self).all[self.id] = self

def update(self):
    """Update the table row corresponding to the current Employee object."""
    sql = """
        UPDATE employees
        SET name = ?, job_title = ?, department_id = ?
        WHERE id = ?
    """
    CURSOR.execute(sql, (self.name, self.job_title,
                        self.department_id, self.id))
    CONN.commit()

@classmethod
def create(cls, name, job_title, department_id):
    """ Initialize a new Employee object and save the object to the database """
    employee = cls(name, job_title, department_id)
    employee.save()
    return employee

```

The final change to the `Employee` class is to update the `instance_from_db` method to include the department id from the table row.

```

@classmethod
def instance_from_db(cls, row):
    """Return an Employee object having the attribute values from the table row."""

```

```

# Check the dictionary for existing instance using the row's primary key
employee = cls.all.get(row[0])
if employee:
    # ensure attributes match row values in case local instance was modified
    employee.name = row[1]
    employee.job_title = row[2]
    employee.department_id = row[3]
else:
    # not in dictionary, create new instance and add to dictionary
    employee = cls(row[1], row[2], row[3])
    employee.id = row[0]
    cls.all[employee.id] = employee
return employee

```

Updating the **Department** class to compute associated **Employee** instances

Since **Department** is not on the owning side of the relationship, we won't store anything about employees in the "departments" table. But we might want to get a list of all employees that work for a department. We'll add a method to the **Department** class that accomplishes this task:

- query the "employees" table for rows that contain a foreign key value that matches the current department's id.
- map the row data to an **Employee** instance.

Note that we import the **Employee** class within the function definition to issues with circular imports (since the **Employee** class imports the **Department** class as well).

```
from __init__ import CURSOR, CONN
```

```
class Department:
```

```
    # existing attributes and methods ...
```

```
def employees(self):
    """Return list of employees associated with current department"""
    from employee import Employee
    sql = """
        SELECT * FROM employees
        WHERE department_id = ?
    """
    CURSOR.execute(sql, (self.id,))

    rows = CURSOR.fetchall()
    return [
        Employee.instance_from_db(row) for row in rows
    ]
```

Exploring the one-to-many relationship

The file `lib/debug.py` has been updated to initialize the database with two departments and several employees. The code generates fake names for the employees, and picks a random job title and department for each employee.

Run the file to recreate the database with sample data:

```
python lib/debug.py
```

In the `ipdb` session, you can explore the table data using the ORM methods:

```
ipdb> Department.get_all()
[<Department 1: Payroll, Building A, 5th Floor>, <Department 2: Human Resources, Building C, East Wing>]

ipdb> Employee.get_all()
```



```
[<Employee 1: Amir, Accountant, Department ID: 1>, <Employee 2: Bola, Manager, Department ID: 1>, <Employee 3: Charl
```

An employee works in one department. Let's get the first employee:

```
ipdb> employee = Employee.find_by_id(1)
ipdb> employee
<Employee 1: Amir, Accountant, Department ID: 1>
```

We can use the employee `department_id` value to get the single associated `Department` instance:

```
ipdb> Department.find_by_id(employee.department_id)
<Department 1: Payroll, Building A, 5th Floor>
```

A department may have many employees. Let's select the payroll department:

```
ipdb> payroll = Department.find_by_id(1)
ipdb> payroll
<Department 1: Payroll, Building A, 5th Floor>
```

We can call the `employees()` method to get the list of employees that work in the payroll department.

```
ipdb> payroll.employees()
[<Employee 1: Amir, Accountant, Department ID: 1>, <Employee 2: Bola, Manager, Department ID: 1>]
```

You can also use the SQLITE EXPLORER to view the relationship.

Testing the `Employee` and `Department` classes

The `lib/testing` folder contains the files `department_orm_test.py` and `employee_orm_test.py` to test the `Department` and `Employee` classes. Run the tests and confirm they pass:

```
pytest -x
```

Conclusion

In this lesson we've seen how to persist a one-to-many relationship between classes. The class or entity on the "many" side of the relationship is responsible for storing and maintaining the relationship, since it only needs to store a single value to reference the associated object/entity on the "one" side.

Solution Code

```
# lib/department.py
from __init__ import CURSOR, CONN

class Department:

    # Dictionary of objects saved to the database.
    all = {}

    def __init__(self, name, location, id=None):
        self.id = id
        self.name = name
        self.location = location

    def __repr__(self):
        return f"<Department {self.id}: {self.name}, {self.location}>"

    @classmethod
    def create_table(cls):
        """ Create a new table to persist the attributes of Department instances """
```

```
sql = """
    CREATE TABLE IF NOT EXISTS departments (
        id INTEGER PRIMARY KEY,
        name TEXT,
        location TEXT)
    """
```

```
CURSOR.execute(sql)
CONN.commit()
```

```
@classmethod
```

```
def drop_table(cls):
```

```
    """ Drop the table that persists Department instances """
```

```
    sql = """
```

```
        DROP TABLE IF EXISTS departments;
    """
```

```
CURSOR.execute(sql)
```

```
CONN.commit()
```

```
def save(self):
```

```
    """ Insert a new row with the name and location values of the current Department instance.
    Update object id attribute using the primary key value of new row.
    Save the object in local dictionary using table row's PK as dictionary key"""
```

```
    sql = """
```

```
        INSERT INTO departments (name, location)
```

```
        VALUES (?, ?)
```

```
    """
```

```
CURSOR.execute(sql, (self.name, self.location))
```

```
CONN.commit()
```

```
self.id = CURSOR.lastrowid
```

```
type(self).all[self.id] = self
```

```
@classmethod
def create(cls, name, location):
    """ Initialize a new Department instance and save the object to the database """
    department = cls(name, location)
    department.save()
    return department

def update(self):
    """Update the table row corresponding to the current Department instance."""
    sql = """
        UPDATE departments
        SET name = ?, location = ?
        WHERE id = ?
    """
    CURSOR.execute(sql, (self.name, self.location, self.id))
    CONN.commit()

def delete(self):
    """Delete the table row corresponding to the current Department instance,
    delete the dictionary entry, and reassign id attribute"""

    sql = """
        DELETE FROM departments
        WHERE id = ?
    """

    CURSOR.execute(sql, (self.id,))
    CONN.commit()

    # Delete the dictionary entry using id as the key
    del type(self).all[self.id]

    # Set the id to None
```

```
self.id = None
```

```
@classmethod
```

```
def instance_from_db(cls, row):
```

```
    """Return a Department object having the attribute values from the table row."""
```

```
    # Check the dictionary for an existing instance using the row's primary key
```

```
    department = cls.all.get(row[0])
```

```
    if department:
```

```
        # ensure attributes match row values in case local instance was modified
```

```
        department.name = row[1]
```

```
        department.location = row[2]
```

```
    else:
```

```
        # not in dictionary, create new instance and add to dictionary
```

```
        department = cls(row[1], row[2])
```

```
        department.id = row[0]
```

```
        cls.all[department.id] = department
```

```
    return department
```

```
@classmethod
```

```
def get_all(cls):
```

```
    """Return a list containing a Department object per row in the table"""
```

```
    sql = """
```

```
        SELECT *
```

```
        FROM departments
```

```
    """
```

```
    rows = CURSOR.execute(sql).fetchall()
```

```
    return [cls.instance_from_db(row) for row in rows]
```

```
@classmethod
```

```
def find_by_id(cls, id):
```

```
"""Return a Department object corresponding to the table row matching the specified primary key"""
```

```
sql = """
    SELECT *
    FROM departments
    WHERE id = ?
    """
```

```
row = CURSOR.execute(sql, (id,)).fetchone()
return cls.instance_from_db(row) if row else None
```

```
@classmethod
```

```
def find_by_name(cls, name):
```

```
    """Return a Department object corresponding to first table row matching specified name"""
```

```
sql = """
    SELECT *
    FROM departments
    WHERE name is ?
    """
```

```
row = CURSOR.execute(sql, (name,)).fetchone()
return cls.instance_from_db(row) if row else None
```

```
def employees(self):
```

```
    """Return list of employees associated with current department"""
```

```
from employee import Employee
```

```
sql = """
    SELECT * FROM employees
    WHERE department_id = ?
    """
```

```
CURSOR.execute(sql, (self.id,))
```

```
rows = CURSOR.fetchall()
```

```
return [
```

```
        Employee.instance_from_db(row) for row in rows
    ]

# lib/employee.py
from __init__ import CURSOR, CONN
from department import Department

class Employee:

    # Dictionary of objects saved to the database.
    all = {}

    def __init__(self, name, job_title, department_id, id=None):
        self.id = id
        self.name = name
        self.job_title = job_title
        self.department_id = department_id

    def __repr__(self):
        return (
            f"<Employee {self.id}: {self.name}, {self.job_title}, " +
            f"Department ID: {self.department_id}>"
        )

    @classmethod
    def create_table(cls):
        """ Create a new table to persist the attributes of Employee instances """
        sql = """
        CREATE TABLE IF NOT EXISTS employees (
            id INTEGER PRIMARY KEY,
            name TEXT,
            job_title TEXT,
```

```

        department_id INTEGER,
        FOREIGN KEY (department_id) REFERENCES departments(id))
    """

    CURSOR.execute(sql)
    CONN.commit()

@classmethod
def drop_table(cls):
    """ Drop the table that persists Employee instances """
    sql = """
        DROP TABLE IF EXISTS employees;
    """
    CURSOR.execute(sql)
    CONN.commit()

def save(self):
    """ Insert a new row with the name, job title, and department id values of the current Employee object.
    Update object id attribute using the primary key value of new row.
    Save the object in local dictionary using table row's PK as dictionary key"""
    sql = """
        INSERT INTO employees (name, job_title, department_id)
        VALUES (?, ?, ?)
    """

    CURSOR.execute(sql, (self.name, self.job_title, self.department_id))
    CONN.commit()

    self.id = CURSOR.lastrowid
    type(self).all[self.id] = self

def update(self):
    """Update the table row corresponding to the current Employee instance."""
    sql = """

```



```
        UPDATE employees
        SET name = ?, job_title = ?, department_id = ?
        WHERE id = ?
    """
    CURSOR.execute(sql, (self.name, self.job_title,
                        self.department_id, self.id))
    CONN.commit()

def delete(self):
    """Delete the table row corresponding to the current Employee instance,
    delete the dictionary entry, and reassign id attribute"""

    sql = """
        DELETE FROM employees
        WHERE id = ?
    """

    CURSOR.execute(sql, (self.id,))
    CONN.commit()

    # Delete the dictionary entry using id as the key
    del type(self).all[self.id]

    # Set the id to None
    self.id = None

@classmethod
def create(cls, name, job_title, department_id):
    """ Initialize a new Employee instance and save the object to the database """
    employee = cls(name, job_title, department_id)
    employee.save()
    return employee
```

```
@classmethod
def instance_from_db(cls, row):
    """Return an Employee object having the attribute values from the table row."""

    # Check the dictionary for existing instance using the row's primary key
    employee = cls.all.get(row[0])
    if employee:
        # ensure attributes match row values in case local instance was modified
        employee.name = row[1]
        employee.job_title = row[2]
        employee.department_id = row[3]
    else:
        # not in dictionary, create new instance and add to dictionary
        employee = cls(row[1], row[2], row[3])
        employee.id = row[0]
        cls.all[employee.id] = employee
    return employee
```

```
@classmethod
def get_all(cls):
    """Return a list containing one Employee object per table row"""
    sql = """
        SELECT *
        FROM employees
    """

    rows = CURSOR.execute(sql).fetchall()

    return [cls.instance_from_db(row) for row in rows]
```

```
@classmethod
def find_by_id(cls, id):
    """Return Employee object corresponding to the table row matching the specified primary key"""
```

```
sql = """
    SELECT *
    FROM employees
    WHERE id = ?
    """

row = CURSOR.execute(sql, (id,)).fetchone()
return cls.instance_from_db(row) if row else None
```

```
@classmethod
def find_by_name(cls, name):
    """Return Employee object corresponding to first table row matching specified name"""
    sql = """
        SELECT *
        FROM employees
        WHERE name is ?
        """

    row = CURSOR.execute(sql, (name,)).fetchone()
    return cls.instance_from_db(row) if row else None
```

This tool needs to be loaded in a new browser window

Load Mapping Object Relationships : Code-Along (CodeGrade) in a new window